Cloud DLP: A Lightweight Rule-Based Data Leakage Prevention System for Cloud Uploads

Author Name[1]
[1]Department of Computer Science, Your University
author@university.edu

Abstract

Organizations moving workloads to the cloud face increased risk of accidental or malicious exposure of sensitive data. We present *Cloud DLP*, a lightweight, rule-based Data Leakage Prevention (DLP) system that inspects file uploads before they reach cloud storage. The system validates file size and type, scans content against configurable regex policies to detect personally identifiable information (PII) and credentials, and persists a full audit trail of allowed, blocked, and failed uploads. Implemented as a REST API backend with a web dashboard, Cloud DLP is designed for easy integration with existing storage pipelines and for extension to production backends (e.g., S3/MinIO). We describe the architecture, implementation, and design choices, and report on correctness of the detection engine via unit tests. The prototype demonstrates that a minimal, policy-driven DLP layer can effectively reduce exposure of sensitive data at upload time while remaining simple to deploy and maintain.

1 Introduction

Cloud adoption has made it common for users to upload files directly to object storage or through web applications. Without inspection, sensitive data—such as email addresses, phone numbers, or passwords—can be stored in plain sight, leading to compliance violations and breach risk. Data Leakage Prevention (DLP) systems aim to detect and block such content before it is persisted.

We present *Cloud DLP*, a lightweight DLP system that sits in front of cloud uploads. It performs three main functions: (1) validation of file size and MIME type, (2) content scanning using configurable regex-based policies to detect PII and credential patterns, and (3) persistent audit logging of every upload outcome. Files that pass validation and scanning are forwarded to (simulated) cloud storage; others are blocked with a clear reason. The design prioritizes simplicity and auditability so that it can be integrated into existing pipelines or extended with stronger detection (e.g., ML-based classifiers) and real cloud backends.

The contributions of this work are: (a) an open, modular architecture for upload-time DLP with a REST API and a web dashboard, (b) a policy-driven detection engine with configurable rules (email, phone, password patterns), (c) consistent audit logging with proper handling of errors and HTTP semantics, and (d) a working prototype and test suite that can serve as a baseline for further research or deployment.

The rest of the paper is organized as follows. Section 2 briefly reviews related work. Section 3 describes the system design and data flow. Section 4 details the implementation. Section 5 presents a small evaluation. Section 6 concludes.

2 Related Work and Background

DLP solutions range from enterprise suites (e.g., endpoint or network DLP) to cloud-native services (e.g., CSP-native classification and policies). Many rely on a combination of pattern matching, keyword lists, and machine learning for document classification and PII detection [1, 2].

Our focus is on a minimal, rule-based layer at the upload path: we do not aim to replace full-featured DLP but to provide a transparent, auditable gate that can be extended.

Regex-based policies are widely used for structured PII (email, phone, credit card) and are easy to tune per jurisdiction. Cloud DLP follows this approach and keeps rules in a single JSON configuration file so that operators can add or modify patterns without code changes. Similar ideas appear in open-source and commercial DLP offerings; our contribution is the integrated design (validation, scan, storage, logging) and the open prototype with a clear API and dashboard.

3 System Design and Architecture

3.1 High-Level Data Flow

The system follows a linear pipeline for each upload.

1. **Upload.** The client (web dashboard or any HTTP client) sends the file via `POST /upload` (multipart/form-data).
2. **Validation.** The server checks file size (e.g., ≤ 5 MB) and MIME type (e.g., `text/plain`, `application/json`, `text/csv`, `application/pdf`). If validation fails, the request is rejected with HTTP 400 and the event is logged as ERROR.
3. **DLP scan.** File content is decoded as text (non-UTF-8 handled with error replacement) and scanned against a set of regex rules. Each rule is named (e.g., `email`, `phone`, `password`); if any rule matches, the upload is *blocked* and the list of matched categories is returned and logged.
4. **Storage.** If the file passes validation and scanning, it is written to the configured storage backend (in the prototype, a local directory simulating cloud storage). On storage failure, the server returns HTTP 500 and logs the error.
5. **Logging.** Every outcome—ALLOWED, BLOCKED, or ERROR—is written to a persistent audit log (SQLite) with filename, status, reason, and UTC timestamp.

3.2 Component Overview

- **API layer (`main.py`).** FastAPI application exposing `/upload`, `/logs`, and `/health`. It enforces size/MIME checks, invokes the DLP engine, calls the storage service, and uses the logger for every outcome. Client errors (validation) return 400; server errors (scan or storage) return 500, with a JSON body `{ "detail": { "status", "reason" } }` for the frontend.
- **Database (`database/models.py`).** SQLAlchemy ORM with an `UploadLog` model (id, filename, status, reason, timestamp). Sessions are managed via a context manager so that commit/rollback and connection close always run, avoiding leaks on exceptions.
- **DLP engine (`dlp_engine/`).** A `DLPDetector` class loads regex rules from `policies.json` and exposes `scan(text)` returning a list of matched rule names. Rules are key–value pairs (e.g., `email`, `phone`, `password`) with regex patterns.
- **Services.** `logger`: appends one row to `upload_logs`. `storage`: writes file bytes to the configured path (local directory in the prototype; replaceable by S3/MinIO).
- **Frontend.** Static HTML/CSS/JS dashboard: file picker and drag-and-drop, display of scan result (ALLOWED/BLOCKED/ERROR), and a table of audit logs refreshed periodically. All user-controlled content is escaped to prevent XSS.

3.3 Design Choices

Session handling. All database access uses a `get_db_session()` context manager that commits on success and rolls back on exception, then always closes the session. This avoids connection leaks and keeps the audit log consistent.

Timestamps. Log timestamps are stored in UTC using `datetime.now(timezone.utc)` so that ordering and reporting are timezone-safe.

Error semantics. Validation failures are treated as client errors (400); scan or storage failures as server errors (500). In all error cases the event is still written to the audit log so that operators have a complete trail.

4 Implementation

4.1 Detection Policies

The prototype ships with three rule categories, defined in `policies.json`:

- **email:** Pattern `[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+` for typical email addresses.
- **password:** Case-insensitive `(password|pwd)[\s:=]+.{6,}` to catch lines like `password: secret123`.
- **phone:** `\b[6-9][0-9]{9}\b` for 10-digit Indian-style mobile numbers.

Adding or changing rules does not require code changes; only the JSON file is edited. The detector compiles each pattern and runs `re.search` over the decoded text; multiple matches (e.g., both email and phone) are reported as a list.

4.2 API and Frontend

The REST API is implemented with FastAPI. The upload handler reads the file into memory, runs validation and `detector.scan()`, then either returns BLOCKED/ALLOWED or raises `HTTPException` with the appropriate status and a JSON detail body. The frontend checks `response.ok` and, for non-OK responses, displays `detail.reason` so that users see clear error messages. Logs are rendered in a table with HTML escaping to prevent injection.

4.3 Testing

The DLP engine is covered by unit tests (pytest): safe text (no findings), single-category hits (email, phone, password), and multiple findings in one text. All tests pass; the suite can be extended to API or integration tests.

5 Evaluation

We focus on correctness of the detection engine and the robustness of the pipeline.

Detection correctness. The five unit tests verify that (1) text with no PII yields an empty list, (2) an email substring is detected as `email`, (3) a 10-digit number matching the phone pattern is detected as `phone`, (4) a line containing `password: ...` is detected as `password`, and (5) a string containing both email and phone yields both labels. This ensures that the policy loading and regex application behave as intended.

Pipeline behavior. Validation (size, MIME) and error handling (session lifecycle, logging on all outcomes, HTTP 400/500) have been implemented and tested manually. The audit log

contains every upload attempt with status and reason, and the dashboard displays them with correct escaping.

Limitations. The current rules are regex-only and may have false positives/negatives (e.g., phone pattern is region-specific). No machine learning or document-level classification is used. Performance on very large files is limited by in-memory decoding and single-threaded scan; for production, streaming and async storage would be considered.

6 Conclusion

We presented Cloud DLP, a lightweight rule-based DLP system for cloud uploads. It validates file size and type, scans content against configurable regex policies for PII and credentials, and maintains a full audit log of allowed, blocked, and failed uploads. The design emphasizes simplicity, auditability, and easy extension to real cloud storage and richer detection. The prototype, including REST API, dashboard, and tests, is available and can serve as a baseline for further research or deployment in controlled environments.

References

[1] M. Gupta, J. Walp, and R. Sharman, "Managing the risk of data leakage in the cloud," in Proc. IEEE Int. Conf. Cloud Comput., 2014, pp. 601–608.

[2] Cloud Security Alliance, "Data Loss Prevention in the Cloud," 2019. [Online]. Available: https://cloudsecurityalliance.org